Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Petrópolis -Rio de Janeiro
2026

*

Jean Victor Borges de Abreu

Pseudo Aleatoriedade e Segurança Criptográfica: Uma Análise da Transição para Algoritmos Pós-Quânticos

Trabalho de Conclusão de Curso apresentado à

FACULDADE DE EDUCAÇÃO TECNOLÓGICA DO ESTADO DO RIO DE JANEIRO
FAETERJ/PETRÓPOLIS

Orientador: Nome do Orientador

Petrópolis -Rio de Janeiro
2026

1 Resumo

2 Abstract

3 Lista de Figuras

4 Lista de Abreviaturas

FAETERJ: Faculdade de Educação Tecnológica do Estado do Rio de Janeiro
TRNG True Random Number Generator

Sumário

1 RESUMO 3

2 ABSTRACT 4

3 LISTA DE FIGURAS 5

4 LISTA DE ABREVIATURAS 6

Sumário 6

5 INTRODUÇÃO 9
5.1 Contextualização do Tema 9
5.2 Justificativa/Problema de Pesquisa 9
5.3 Motivação/Hipotese e Ameaca Tecnológica 9
5.4 Objetivos 9
5.5 Metodologia 9

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO . 10
6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade 10
6.2 Entropia e Fontes de Incerteza 10
6.3 A Semente (Seed) como Elemento Crítico 11
6.4 Avaliação Estatística da Aleatoriedade 11
6.4.1 Bateria de Testes 12
6.4.1.1 Random 13
6.4.1.2 SegureRandom 13
6.4.2 Teste de Frequência 14
6.4.2.1 SegureRandom 14

7 GERADORES CRIPTOGRAFICAMENTE SEGUROS 17
7.1 PRNGs vs. CSPRNGs 17
7.2 O Ataque do Próximo Bit 17

7.2.1 Implementação Prática (Demonstração) 17
7.3 Implementação de CSPRNGs em Java 17
7.4 Falhas Históricas de Aleatoriedade 17

8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL 18
8.1 Conceitos Fundamentais de Computação Quântica 18
8.2 Algoritmo de Shor 18
8.3 Algoritmo de Grover 18
8.4 Harvest Now, Decrypt Later 18

9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC) 19
9.1 Definição e Objetivos da PQC 19
9.2 Paradigmas Matemáticos da PQC 19
9.3 Criptografia Baseada em Reticulados 19
9.4 Learning With Errors (LWE) 19
9.4.1 Implementação Didática em Java (Demonstração) 19
9.5 Padronização Pós-Quântica 19

10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA 20
10.1 Suporte à Criptografia Pós-Quântica no Java 20
10.2 Crypto-Agility 20
10.2.1 Implementação Prática: Demo de Crypto-Agility 20
10.3 Comparativo de Modelos 20
10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber 20

11 DISCUSSÃO E ANÁLISE CRÍTICA 21
11.1 Impacto em Desempenho 21
11.2 Barreiras de Adoção 21
11.3 Implicações Sociais e Éticas 21

12 CONCLUSÃO 22
12.1 Síntese dos Resultados 22
12.2 Resposta ao Problema de Pesquisa 22
12.3 Trabalhos Futuros 22

13 BIBLIOGRAFIAS UTEIS 23

14 COISAS POSSIVELMENTE UTEIS EM ALGUM MOMENTO . . 25

15 CONCLUSÃO 27

16 EXEMPLOS PARA USO DA BIBLIOGRÁFIA 28

REFERÊNCIAS 29

5 Introdução

A onipresença da criptografia na infraestrutura digital moderna (internet, sistemas bancários, IoT e comunicações seguras).

5.1 Contextualização do Tema

Textos

5.2 Justificativa/Problema de Pesquisa

O conflito entre o determinismo computacional e a necessidade de aleatoriedade em sistemas criptográficos.

5.3 Motivação/Hipotese e Ameaca Tecnológica

O impacto da computação quântica — especialmente por meio do Algoritmo de Shor — sobre os modelos atuais de segurança e pseudoaleatoriedade.

5.4 Objetivos

Objetivo Geral: Analisar a evolução dos geradores pseudoaleatórios criptograficamente seguros frente à transição para a criptografia pós-quântica. Objetivos Específicos: Estudar PRNGs e CSPRNGs clássicos Analisar o impacto dos algoritmos quânticos Investigar os novos paradigmas de pseudoaleatoriedade baseados em reticulados Avaliar a adoção desses conceitos no ecossistema Java

5.5 Metodologia

Pesquisa bibliográfica, análise documental de padrões criptográficos e estudo prático no ecossistema Java.

6 FUNDAMENTOS DA ALEATORIEDADE EM COMPUTAÇÃO

6.1 Aleatoriedade Verdadeira vs. Pseudoaleatoriedade

Definições de TRNG e PRNG (Pseudo-Random Number Generator).

TRNG é um dispositivo que gera números aleatórios a partir de um processo físico, e não por meio de um algoritmo. Tais dispositivos são frequentemente baseados em fenômenos microscópicos que geram sinais de ruído estatisticamente aleatórios de baixo nível, como ruído térmico, efeito fotoelétrico, ou fenômenos quânticos. Esses processos estocásticos são, em teoria, completamente imprevisíveis enquanto uma equação que governa tais fenômenos for desconhecida ou incomputável. Isso contrasta com o paradigma de geração de números pseudo- aleatórios comumente implementado através de algoritmos.

PRNG refere-se a uma sequência de números ou dados que aparenta ser aleatória, mas é gerada por um algoritmo determinístico. E é gerada por algoritmos que utilizam fórmulas matemáticas para produzir resultados aparentemente aleatórios. Embora sequências pseudoaleatórias possam imitar a aleatoriedade até certo ponto, elas são, em última análise, determinísticas e repetíveis. Esses algoritmos recebem um valor inicial, chamado semente, e o utilizam para produzir uma sequência de números. Ao aplicar operações matemáticas ao valor da semente, o PRNG produz uma série de números aparentemente aleatórios. O valor da semente pode ser escolhido arbitrariamente, mas afeta significativamente a sequência resultante de números pseudoaleatórios.

6.2 Entropia e Fontes de Incerteza

Captação de eventos físicos imprevisíveis como base para sistemas criptográficos seguros.

Entropia é uma medida da aleatoriedade ou imprevisibilidade dos dados. No contexto da criptografia, a entropia é usada para gerar números aleatórios ou chaves que são essenciais para a comunicação segura e a criptografia. Sem uma boa fonte de entropia, os protocolos criptográficos podem se tornar vulneráveis a ataques que exploram a previsibilidade das chaves geradas.

Na maioria dos sistemas operacionais, a entropia é gerada pela coleta de eventos aleatórios de diversas fontes, como interrupções de hardware, movimentos do mouse, pressionamentos de teclas e atividade do disco. Esses eventos são inseridos em um conjunto de entropia, que é então usado para gerar números aleatórios quando necessário.

Protocolos criptográficos, como SSL/TLS, SSH e IPsec, dependem fortemente da entropia para gerar números aleatórios e chaves para criptografia, autenticação e proteção

de integridade. A aleatoriedade desses números é crucial para a segurança da comunicação. Se um atacante conseguir prever os números aleatórios usados em um protocolo criptográfico, ele poderá comprometer a confidencialidade, a integridade ou a autenticidade da comunicação. Se a chave mestra prévia for previsível ou tendenciosa, um atacante poderá interceptar a mensagem criptografada, descriptografá-la e obter acesso às chaves de sessão. Isso permitiria ao atacante espionar a comunicação, modificar os dados ou se passar pelo servidor.

Um tipo de entropia. É a entropia de "Shannon". Que utiliza umm tratamento probabilístico de uma função logarítmica. Sendo sua formula:

$$\boldsymbol{H(X) = -\sum_{i=1}^{n} P(x_i) \log_b P(x_i)} \tag{6.1}$$

Em que:

- $\boldsymbol{H(X)}$ é a entropia em bits;
- $\boldsymbol{p_i}$ é a probabilidade de ocorrência do resultado $\boldsymbol{i}$;
- $\boldsymbol{n}$ é o número total de resultados possíveis.

Uma limitação determinística é em cima de um processador de maquina de estados finitos. Se acaso for utilizado as mesmas variáveis, o resultado permanece inalterado. Onde se um invasor tiver a certeza da seed de origem da criptografia. Ela por si mesma, não é aleatória. Para evitar tais acontecimentos se faz necessário que o software faça a captação da incerteza do mundo físico.

[Criptografia Simétrica]

Se trata da utilização da mesma chave para o processo de encriptação e desencriptaçao. Apesar da primitiva de chave simétrica embarcar não só encriptadores, como também funções hash de tamanho arbitrário como os MACs (Message Authentication Codex), sistemas de assinatura, sequencias pseudoaleatórias e primitivas de identificação.

6.3 A Semente (Seed) como Elemento Crítico

O papel do valor inicial na previsibilidade e segurança de sequências pseudoaleatórias.

6.4 Avaliação Estatística da Aleatoriedade

Bateria de testes SP 800-22 do NIST e o conceito de indistinguibilidade estatística.

O NIST Test Suite consiste em um pacote de 15 testes de estatistas desenvolvidos para avaliar a as sequencias binarias aleatórias (com comprimento arbitrário), gerados por geradores de números aleatórios ou pseudo criptografados, seja de forma em hardware ou

software. les podem ser decompostos em vários subtestes de acordo com a necessidade e o tipo de geração aleatória que podem ter uma sequencia. Os 15 testes são:

1. Teste de Frequência (monobit)
2. Teste de Frequência dentro de um bloco
3. Teste de Sequencias
4. Testes para Maior Sequencia de Uns em um bloco
5. Testes de Posto de Matriz Binária
6. Teste da Transformada Discreta ded Fourier (Espectral)
7. Teste de Correspondência ded Modelos não Sobrepostos
8. Teste de Correspondência de Modelos Sobrepostos
9. Teste "Estatístico Universal"de Maurer
10. Teste de Complexidade Linear
11. Teste Serial
12. Teste de Entropia Aproximada
13. Teste de Somas Cumulativas (Cusums)
14. Teste de Excursões Aleatórias
15. Teste de Variante de Excursões Aleatórias

Falar separadamente sobre cada um dos 15 testes. Mostrando um teste local separado. (Seguir documentacao do guthub para execucao local)(GITHUB, s.d.)

6.4.1 Bateria de Testes

Os arquivos binários gerados em Java servem como a entrada bruta para o NIST SP 800–22, que interpreta os bytes como fluxos contínuos de bits para análise estatística. Ao executar o programa, a ferramenta particiona esses dados em blocos de tamanho fixo, como sequências de $\mathbf{10^6}$ bits, aplicando sobre cada um deles quinze testes que buscam padrões não aleatórios, como repetições excessivas ou falta de complexidade linear. Para cada teste, o sistema calcula um $\boldsymbol{P}$-value, que quantifica a probabilidade de a sequência ter sido gerada por um processo puramente aleatório. Ao final, o sucesso do gerador é determinado pela uniformidade desses valores e pela proporção de sequências que mantiveram o $\boldsymbol{P}$-value $\geq \mathbf{0,01}$.

Figura 1 – Modelo simples de um PRNG com LCG

```java
import java.io.BufferedOutputStream;
import java.io.FileOutputStream;
import java.io.IOException;
import java.security.SecureRandom;
import java.util.Random;

public class Main {
Run | Debug
public static void main(String[] args) {
        int tamanhoBytes = 12_500_000;

        gerarInseguro(nomeArquivo: "inseguro.bin", tamanhoBytes);
        gerarSeguro(nomeArquivo: "seguro.bin", tamanhoBytes);
    }

    public static void gerarInseguro(String nomeArquivo, int tamanho) {
        Random prng = new Random();
        byte[] buffer = new byte[1024];

        try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(nomeArquivo))) {
            for (int i = 0; i < tamanho; i += buffer.length) {
                prng.nextBytes(buffer);
                bos.write(buffer);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

    public static void gerarSeguro(String nomeArquivo, int tamanho) {
        SecureRandom csprng = new SecureRandom();
        byte[] buffer = new byte[1024];

        try (BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(nomeArquivo))) {
            for (int i = 0; i < tamanho; i += buffer.length) {
                csprng.nextBytes(buffer);
                bos.write(buffer);
            }
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}
```

Fonte: Elaborado pelo autor (2026)

6.4.1.1 Random

A funćao java.util.Random faz uso do modelo LCG "Gerador Congruencial Linear". Ele é um modelo puramente matemático. Onde o mesmo é usado para gerar uma sequência de números aleatórios, calculados por uma equação linear.

$$\boldsymbol{x_{i+1} = (A \cdot x_i + C) \pmod{M}} \tag{6.2}$$

Onde o valor inicial "x0" é a seed, A e C são parâmetros inteiros que podem ter um valor já pré-definido ou passados diretamente, M é o espaço dos valores aleatórios possíveis e "mod" é o restante da divisão inteira.

6.4.1.2 SegureRandom

A funćao java.security.SegureRandom faz utilizaćao do CSPRNG, que diferente do LCG. Esse tipo de gerador busca caos do hardware. Sendo elas interrupćoes de hardware, pressionar das teclas do teclado, posićao e movimento do mouse, pequenas flutuaćoes de voltagem do processador e/ou o tempo que o disco demora pra fazer a requisićao. Esses eventos imprevisíveis são misturados, usando funćoes hash como SHA-256 para criar uma

seed de alta entropia. Mesmo que se conheća o algoritmo, não é possivel recriar o mesmo evento. Justamente pelo fator caos fisico que foi utilizado.

6.4.2 Teste de Frequência

6.4.2.1 SegureRandom

O teste de frequência (monobit) é o teste mais básico da bateria do NIST. É o pilar para a execućão dos demais testes. Se um gerador não for capaz de equilibrar a quantidade de bits '1' e '0' em uma sequência, ele falha nesse teste fundamental, indicando que a sequência não é aleatória o suficiente para aplicações criptográficas. Verificaćão do resultado do teste de frequência (monobit) para ambos os geradores, destacando a falha do java.util.Random e o sucesso do SecureRandom.

Figura 2 – Resultado do Teste de Frequência (Monobit) para SecureRandom

Projetos > NIST > sts-2.1.2 > experiments > AlgorithmTesting > Frequency > results.txt

1	0.278365	0.715858	0.444867	0.839917	0.578211	0.151572	0.179596	0.035729	0.253454	0.116881
2	0.852445	0.102259	0.813433	0.148177	0.556532	0.296485	0.459300	0.250969	0.357573	0.433040
3	0.648390	0.541862	0.357573	0.885500	0.013740	0.652710	0.855583	0.920344	0.784085	0.838353
4	0.351337	0.548506	0.801041	0.590577	0.030159	0.777944	0.939419	0.917169	0.420243	0.374541
5	0.151002	0.398669	0.307728	0.838353	0.608651	0.333044	0.387588	0.662837	0.274833	0.705431
6	0.022021	0.096513	0.587819	0.925109	0.336050	0.033670	0.079431	0.619894	0.215716	0.206949
7	0.060381	0.013212	0.396438	0.486429	0.354446	0.961716	0.106527	0.458087	0.000128	0.551175
8	0.124538	0.471525	0.964904	0.708404	0.336050	0.976067	0.907653	0.376696	0.934647	0.409940
9	0.596112	0.381030	0.446060	0.252624	0.369186	0.969688	0.129014	0.744424	0.453255	0.142104
10	0.611453	0.536575	0.883921	0.419091	0.258476	0.705431	0.985639	0.850877	0.966499	0.469066

Fonte: Elaborado pelo autor (2026)

Cada valor apresentado no resultado do teste de frequência (monobit) representa o ***P***-value calculado para a sequência de bits gerada. Ele verifica se a quantidade de bits '1' e '0' está equilibrada, como seria esperado em uma sequência verdadeiramente aleatória.

P-value proximo de 1 indica que possui uma distribuição equilibrada, como é o caso do valor 0.985639.

P-value proximo de 0 indica que possui uma distribuição desequilibrada, como é o caso do valor 0.022021.

P-value menor que 0.01 é considerado uma falha no teste, indicando que a sequência não é aleatória o suficiente, como é o caso do valor 0.000128.

Figura 3 – Resultado final do Teste de Frequência (Monobit) para SecureRandom

```
1  ------------------------------------------------------------------------------
2  RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES
3  ------------------------------------------------------------------------------
4     generator is <seguro.bin>
5  ------------------------------------------------------------------------------
6  C1  C2  C3  C4  C5  C6  C7  C8  C9 C10  P-VALUE  PROPORTION  STATISTICAL TEST
7  ------------------------------------------------------------------------------
8  10  10   9  15  12   9   6   7  10  12  0.739918     99/100     Frequency
```

Fonte: Elaborado pelo autor (2026)

Os numeros apresentados no resultado final do teste de frequência (monobit). Da coluna C1 até a coluna C10 representam a distribuićao das 100 amostras. Que foi dividida em 10 blocos de 10 amostras cada. O valor da coluna P representa a proporção de amostras que passaram no teste, ou seja, que tiveram um ***P***-value maior ou igual a 0.01. O valor da coluna P' representa o ***P***-value calculado para a proporção de amostras. Cada amostra tiveram um ***P***-value entre 0.0 e 1.0.

Na coluna C1 nos é apresentado que obtivemos 10 amostras com ***P***-value maior ou igual a 0.01.

Na coluna C10 nos é apresentado que obtivemos 12 amostras com ***P***-value entre 0.9 a 1.00.

O ***P***-value 0.739918 apresentado é um teste Qui-Quadrado aplicado sobre os números das colunas C1 a C10. Fazendo a medićão da uniformidade e a distribuićao dos resultados. Se este valor for maior ou igual a 0.0001, o teste é considerado bem sucedido, indicando que os resultados estão distribuídos de maneira uniforme. Com o resultado de 0.739918, o SegureRandom passou no teste de frequência (monobit) com uma distribuição de ***P***-value que é considerada exelente. Capaz de produzir sequências de bits que se aproximam muito da aleatoriedade ideal, o que é crucial para aplicações criptográficas seguras.

Com uma proporsão de passagem de 99/100. Significa que 99 amostras obtiveram um ***P***-value maior ou igual a 0.01. Para uma amostragem de 100, o minimo requisitado para passar no teste é de 96/100.

Figura 4 – Resultado do Teste de Frequência (Monobit) para Random

Projetos > NIST > sts-2.1.2 > experiments > AlgorithmTesting > Frequency > results.txt

0.091796	0.714365	0.579579	0.945786	0.150434	0.430697	0.175736	0.214237	0.304892	0.923521
0.411077	0.833668	0.598888	0.777944	0.918757	0.014286	0.016666	0.741400	0.303010	0.824314
0.787160	0.752003	0.933056	0.814985	0.352371	0.885500	0.968093	0.036439	0.733857	0.858723
0.730846	0.709893	0.753521	0.879187	0.137809	0.470294	0.921932	0.492713	0.605854	0.587819
0.557876	0.030773	0.835229	0.662837	0.322174	0.515692	0.791780	0.277479	0.436569	0.085432
0.198443	0.516985	0.519575	0.944194	0.797951	0.480188	0.735363	0.625550	0.644081	0.359667
0.162113	0.048155	0.977662	0.945786	0.287329	0.083988	0.990426	0.350304	0.279253	0.966499
0.761128	0.070918	0.882343	0.219447	0.156191	0.708404	0.739889	0.819646	0.054606	0.171939
0.160319	0.027524	1.000000	0.020449	0.064892	0.135176	0.537895	0.575479	0.131555	0.651269
0.850877	0.491453	0.411077	0.203373	0.901315	0.810330	0.419091	0.956935	0.018773	0.386496

Fonte: Elaborado pelo autor (2026)

Figura 5 – Resultado final do Teste de Frequência (Monobit) para Random

Projetos > NIST > sts-2.1.2 > experiments > AlgorithmTesting > finalAnalysisReport.txt

```
------------------------------------------------------------------------------
RESULTS FOR THE UNIFORMITY OF P-VALUES AND THE PROPORTION OF PASSING SEQUENCES
------------------------------------------------------------------------------
   generator is <inseguro.bin>
------------------------------------------------------------------------------
 C1  C2  C3  C4  C5  C6  C7  C8  C9 C10  P-VALUE  PROPORTION  STATISTICAL TEST
------------------------------------------------------------------------------
 14  10   6   7   9   9   5  15  11  14  0.275709    100/100     Frequency
```

Fonte: Elaborado pelo autor (2026)

Por mais que olhando os valores apresentados no resultado do teste de frequência (monobit) para o Random, seja possível observar que a maioria dos ***P***-value estão acima de

0.01, o que indicaria uma distribuição equilibrada, O resultado final do teste de frequência (monobit) para o Random mostra que ele obteve um ***P***-value de 0.275709 um valor mais proximo ao 0.0001.

Por mais que seu valor seja inferior ao do SegureRandom, o Random ainda assim passou no teste de frequência (monobit) com uma distribuição de ***P***-value que é considerada boa. Um falso positivo, devido a maneira como o Random funciona, utilizando o LCG que é matematicamente desenhado para ter uma distribuićao uniforme em complexidades simples.

7 Geradores Criptograficamente Seguros

7.1 PRNGs vs. CSPRNGs

Diferenças arquiteturais e requisitos de segurança.

7.2 O Ataque do Próximo Bit

Por que geradores comuns falham em contextos criptográficos.

7.2.1 Implementação Prática (Demonstração)

Demonstração do ataque contra java.util.Random, mostrando previsão de saídas futuras com base em observações anteriores — contrasta diretamente com o CSPRNG.

7.3 Implementação de CSPRNGs em Java

Análise da classe java.security.SecureRandom e seus provedores (SHA1PRNG, NativePRNG).

7.4 Falhas Históricas de Aleatoriedade

Casos emblemáticos como o Debian OpenSSL e problemas recorrentes em dispositivos IoT.

8 COMPUTAÇÃO QUÂNTICA E A QUEBRA DO PARADIGMA ATUAL

8.1 Conceitos Fundamentais de Computação Quântica

Diferença entre bits clássicos e qubits.

8.2 Algoritmo de Shor

Fatoração eficiente e o colapso dos sistemas baseados em RSA e ECC.

8.3 Algoritmo de Grover

Impacto sobre criptografia simétrica e necessidade de aumento no tamanho das chaves.

8.4 Harvest Now, Decrypt Later

A urgência da migração para algoritmos resistentes a ataques quânticos.

9 CRIPTOGRAFIA PÓS-QUÂNTICA (PQC)

9.1 Definição e Objetivos da PQC

Algoritmos clássicos projetados para resistir a ataques quânticos.

9.2 Paradigmas Matemáticos da PQC

Reticulados, códigos corretores de erro, funções hash e isogenias.

9.3 Criptografia Baseada em Reticulados

Introdução ao conceito de Lattices como base da nova segurança criptográfica.

9.4 Learning With Errors (LWE)

Fundamento matemático da pseudoaleatoriedade pós-quântica.

9.4.1 Implementação Didática em Java (Demonstração)

Implementação simplificada do LWE do zero em Java para fins didáticos, mostrando como o erro intencional funciona matematicamente antes de utilizar a biblioteca pronta.

9.5 Padronização Pós-Quântica

O papel do ML-KEM (Kyber) e o processo de padronização conduzido pelo NIST.

10 A TRANSIÇÃO PARA PQC NO ECOSSISTEMA JAVA

10.1 Suporte à Criptografia Pós-Quântica no Java

Evolução do JDK e integração de algoritmos pós-quânticos em versões modernas (Java 21+).

10.2 Crypto-Agility

Boas práticas para desenvolvimento de sistemas criptográficos flexíveis e preparados para mudanças de algoritmo.

10.2.1 Implementação Prática: Demo de Crypto-Agility

Criação de uma interface Java que abstrai o algoritmo de geração de chaves, permitindo trocar de RSA para Kyber alterando apenas a configuração — sem reescrever o código da aplicação.

10.3 Comparativo de Modelos

Geração de chaves e aleatoriedade no modelo clássico versus modelo baseado em reticulados (Module-Lattice).

10.3.1 Implementação Prática: Benchmark Clássico vs. Kyber

Benchmark comparativo entre java.security.SecureRandom clássico e ML-KEM (Kyber) via Bouncy Castle, medindo tempo de geração de chaves, uso de memória e throughput. Inclui comparativo de tamanho de chaves entre RSA-2048 e Kyber. Nota: Confirmar com o orientador o uso da biblioteca Bouncy Castle como dependência, por ser a opção mais acessível para PQC em Java atualmente.

11 DISCUSSÃO E ANÁLISE CRÍTICA

11.1 Impacto em Desempenho

Custos computacionais, consumo de memória e latência — baseados nos resultados do benchmark da seção 6.3.1.

11.2 Barreiras de Adoção

Sistemas legados, limitações de hardware e resistência organizacional.

11.3 Implicações Sociais e Éticas

Soberania de dados, privacidade, vigilância e assimetria tecnológica.

12 CONCLUSÃO

12.1 Síntese dos Resultados

A inevitabilidade da transição criptográfica.

12.2 Resposta ao Problema de Pesquisa

A evolução da pseudoaleatoriedade como pilar da segurança pós-quântica.

12.3 Trabalhos Futuros

Geradores Quânticos de Números Aleatórios (QRNG) e integração híbrida TRNG + PQC.

13 Bibliografias Uteis

1. Fundamentos e Estado da Arte (Capítulos 2 e 3)

Para falar de aleatoriedade, sementes e o funcionamento dos CSPRNGs atuais:

Livro: The Art of Computer Programming, Vol 2: Seminumerical Algorithms (Donald Knuth).

Onde usar: Capítulos 2.1 e 3.1. É a base matemática para entender geradores lineares e por que eles são previsíveis.

Livro: Handbook of Applied Cryptography (Menezes, van Oorschot e Vanstone).

Onde usar: Capítulo 3.2 e 4.1. Consulte o Capítulo 5 (Pseudorandom Bits and Sequences). Ele define formalmente o "Next Bit Test".

Norma Técnica: NIST SP 800-22 Rev 1a: A Statistical Test Suite for Random and Pseudorandom Number Generators.

Onde usar: Capítulo 2.4. É o documento que dita como o mundo testa se um PRNG é bom.

2. A Ameaça Quântica (Capítulo 4)

Aqui você precisa citar os fundamentos da física quântica aplicados à quebra de códigos:

Livro: Quantum Computation and Quantum Information (Michael Nielsen e Isaac Chuang).

Onde usar: Capítulo 4.1 e 4.2. É a "bíblia"da computação quântica. Essencial para explicar Qubits e o Algoritmo de Shor.

Artigo Seminal: Algorithms for quantum computation: discrete logarithms and factoring (Peter Shor, 1994).

Onde usar: Capítulo 4.2. O artigo que mudou a história da criptografia.

Relatório: NIST IR 8105: Report on Post-Quantum Cryptography.

Onde usar: Capítulo 4.4. Explica o conceito de "Harvest Now, Decrypt Later".

3. Criptografia Pós-Quântica e Reticulados (Capítulo 5)

Esta é a parte mais densa e moderna do seu trabalho:

Artigo de Revisão: A Decade of Lattice Cryptography (Chris Peikert). Onde usar: Capítulo 5.3 e 5.4. Peikert é um dos maiores nomes da área. Este artigo resume como a matemática de reticulados (Lattices) se tornou segura.

Artigo Técnico: The Learning with Errors Problem (Oded Regev). Onde usar: Capítulo 5.4. Oded Regev criou o conceito de LWE. Este artigo é a fonte primária para a sua fundamentação matemática.

Documento Oficial (FIPS): FIPS 203: Module–Lattice–Based Key–Encapsulation Mechanism Standard.

Onde usar: Capítulo 5.5. Este é o padrão final do ML-KEM (Kyber). É a referência mais importante para o seu futuro capítulo 6.

4. Transição e Ecossistema Java (Capítulo 6)

Para a parte teórica sobre Java e Agilidade Criptográfica: Documentação JEP (JDK Enhancement Proposals):

JEP 466 (Class-File API) e documentos relacionados à implementação do ML-KEM no provedor SunJCE.

Onde usar: Capítulo 6.1.

Artigo sobre Crypto-Agility: A Taxonomy of Crypto-Agility (S. Ott et al.).

Onde usar: Capítulo 6.2. Ajuda a definir por que sistemas modernos não podem ficar "presos"a um único algoritmo.

14 Coisas possivelmente Uteis em algum momento

PRNG é uma sigla para “Pseudo Random Number Generator” ou “Gerador de Números Pseudo Aleatórios”. Seu significado é que a sua geração de números não segue uma estrutura aleatória. Ele utiliza um ponto inicial denominado “seed” para dar início a sua geração de números. A seed é um valor base para ser utilizado no PRNG. Utilizando a mesma seed, o resultado final do PRNG será o mesmo.

O PRNG funciona a partir de uma função que recebe a seed por parâmetro. E através de um cálculo, ele retorna o resultado dessa fórmula. Uma das fórmulas mais comuns para esse tipo de gerador de pseudo números aleatórios é no modelo LCG “Gerador Congruencial Linear”

$$\boldsymbol{x_{i+1} = (A \cdot x_i + C) \pmod{M}} \tag{14.1}$$

“Onde o valor inicial "x0"é a seed, A e C são parâmetros inteiros que podem ter um valor já pré-definido ou passados diretamente, M é o espaço dos valores aleatórios possíveis e "mod"é o restante da divisão inteira.”

Figura 6 – Modelo simples de um PRNG com LCG

```java
public class Main {
    public static void main(String[] args) {
        long seed = 12345;
        long a = 1664525;
        long c = 1013904223;
        long mod = (long) Math.pow(2, 32);

        LCG lcg = new LCG(seed, a, c, mod);

        for (int i = 0; i < 10; i++) {
            System.out.println(lcg.next());
        }

    }

    public static class LCG {  2 usages

        private long a;  2 usages
        private long c;  2 usages
        private long mod;  2 usages
        private long xi;  4 usages

        public LCG(long seed, long a, long c, long mod) {  1 usage
            this.xi = seed;
            this.a = a;
            this.c = c;
            this.mod = mod;
        }

        public long next() {  1 usage
            xi = (a * xi + c) % mod;
            return xi;
        }
    }
}
```

Fonte: Elaborado pelo autor (2026)

Mesmo após inúmeras tentativas. Devido a seed permanecer inalterada. O resultado permanece o mesmo em todos os resultados obtidos.

Por mais que tenha se passado um valor fixo na seed no exemplo prático, a seed é definida pelo sistema operacional, esse número é denominado entropia. Que se tem por referência a quantidade de imprecisão de um sistema. Quanto maior o número da entropia, maior é a incerteza. Esse valor de entropia é gerado a partir de certos eventos que ocorrem no momento de sua captura, como por exemplo o movimento do mouse, os inputs no teclado etc. Essas capturas são convertidas em um valor numérico para ser usado na seed.

Esses inputs que o sistema captura podem ser previsíveis, tornando a entropia causada menor. Sendo possível pré-determinar o valor gerado ao final para seed. Apesar das aplicações para o dia a dia ou em pequenos projetos serem o suficiente. Esse método não é seguro para aplicações de alto risco.

Figura 7 – Resultado do script LCG

```
/app/extra/jbr/bin/java -javaagent:/app/extra/lib/idea_rt.jar=34643 -Dfile.encoding=UTF-8 -Dsun.stdout.encoding=UTF-8 -Dsun.stderr.encodin
87628868
71072467
2332836374
2726892157
3908547000
483019191
2129828778
2355140353
2560230508
3364893915

Process finished with exit code 0
```

Fonte: Elaborado pelo autor (2026)

15 Conclusão

Considerações finais.

16 Exemplos para uso da bibliográfia

para livros(KNUTH, 1997)

para artigos(MATSUMOTO; NISHIMURA, 1998)

para sites e outras fontes(Oracle Corporation, 2023)

para outros TCC e afins(ALMEIDA, 2025)

para fichas tecnicas e documentacao(International Organization for Standardization, 2022)

Referências

ALMEIDA, C. *Análise de Geradores de Números Aleatórios em Sistemas Embarcados*. Dissertação (Dissertação (Mestrado em Engenharia Eletrônica)) — Universidade Estadual do Rio de Janeiro, Rio de Janeiro, 2025.

ALVES, P. *Desenvolvimento de TRNG (True Random Number Generator) através de células memristoras*. Dissertação (Trabalho de Conclusão de Gradução) — Universidade Federal do Rio Grande do Sul, Porto Alegre, 2022. Disponível em: <https://lume.ufrgs.br/handle/10183/251751>. Acesso em: 18 fev. 2026.

GITHUB. *NIST*. s.d. Disponível em: <https://github.com/terrillmoore/NIST-Statistical-Test-Suite>. Acesso em: 02 mar. 2026.

International Organization for Standardization. *ISO/IEC 27001: Information technology - Security techniques*. Geneva, CH, 2022.

KNUTH, D. E. *The Art of Computer Programming: Seminumerical Algorithms*. Reading, MA: Addison-Wesley, 1997. v. 2.

Lenovo. *Glossary: Pseudorandom*. s.d. Disponível em: <https://www.lenovo.com/us/en/glossary/pseudorandom>. Acesso em: 18 fev. 2026.

MATSUMOTO, M.; NISHIMURA, T. Mersenne twister: A 623-dimensionally equidistributed uniform pseudo-random number generator. *ACM Transactions on Modeling and Computer Simulation*, v. 8, n. 1, p. 3–30, 1998.

NASCIMENTO, W. S. *Sobre algumas características da entropia de Shannon para sistemas atômicos confinados*. Dissertação (Dissertação (Mestrado em Física)) — Universidade Federal da Bahia Instituto de Física, Bahia, 2013. Disponível em: <https://repositorio.ufba.br/bitstream/ri/28664/1/disserta%c3%a7%c3%a3o_wallas_final.pdf>. Acesso em: 25 fev. 2026.

NETDATA. *Understanding Entropy Key To Secure Cryptography & Randomness*. s.d. Disponível em: <https://www.netdata.cloud/blog/understanding-entropy-the-key-to-secure-cryptography-and-randomness>. Acesso em: 18 fev. 2026.

Oracle Corporation. *Class Random: Java Platform SE 17*. 2023. Disponível em: <https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Random.html>. Acesso em: 18 fev. 2026.

RUKHIN, A. A statistical test suite for random and pseudorandom number generators for cryptographic applications. *National Institute of Standards and Technology*, v. 2, n. 1, p. 23–62, 2010.